

STORED PROCEDURES AND FUNCTIONS

1. Create table and insert values

Firstly, Created the table Employee and Department as we are using stored procedures and functions to fetch data..

The screenshot displays a SQL query window with the following code:

```
3 USE CompanyDB;
4
5 -- Create Department Table
6 CREATE TABLE Department (
7     DeptID INT PRIMARY KEY,
8     DeptName VARCHAR(50)
9 );
10
11 -- Create Employee Table
12 CREATE TABLE Employee (
13     EmpID INT PRIMARY KEY,
14     EmpName VARCHAR(50),
15     Salary DECIMAL(10,2),
16     DeptID INT FOREIGN KEY REFERENCES Department(DeptID)
17 );
18
```

Below the query window, the 'Results' tab shows the data inserted into the tables:

	DeptID	DeptName
1	1	HR
2	2	IT
3	3	Finance

	EmpID	EmpName	Salary	DeptID
1	101	Ross	50000.00	1
2	102	Monica	60000.00	2
3	103	Chandler	55000.00	2
4	104	Rachel	70000.00	3
5	105	Joey	45000.00	1
6	106	Phoebe	75000.00	1

Stored Procedure

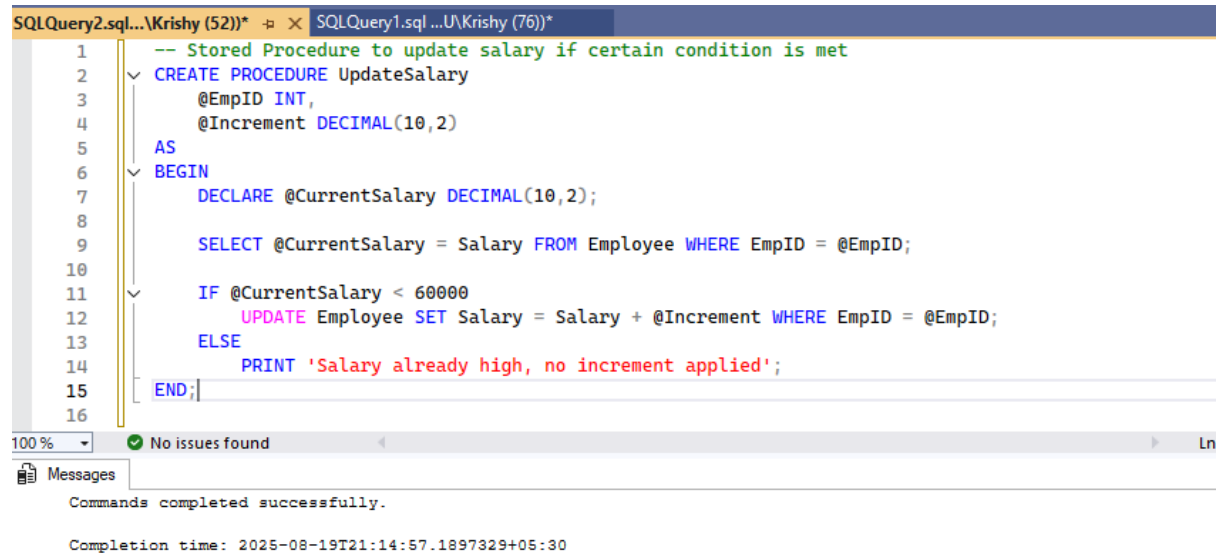
- A block of SQL statements stored in the database.
- Can perform **INSERT**, **UPDATE**, **DELETE** operations.
- May or may not return a value.
- Called with **CALL** or **EXEC**.

Steps to be followed

- Create a procedure
- Declare it
- Call it
- Get the output

Step 1- Create Procedure and declare that

The stored procedure(UpdateSalary) is created for update the salary and we used if else statement inorder to update that.



```
1  -- Stored Procedure to update salary if certain condition is met
2  CREATE PROCEDURE UpdateSalary
3      @EmpID INT,
4      @Increment DECIMAL(10,2)
5  AS
6  BEGIN
7      DECLARE @CurrentSalary DECIMAL(10,2);
8
9      SELECT @CurrentSalary = Salary FROM Employee WHERE EmpID = @EmpID;
10
11     IF @CurrentSalary < 60000
12         UPDATE Employee SET Salary = Salary + @Increment WHERE EmpID = @EmpID;
13     ELSE
14         PRINT 'Salary already high, no increment applied';
15 END;
```

100 % No issues found

Messages

Commands completed successfully.

Completion time: 2025-08-19T21:14:57.1897329+05:30

Step 2- Call Procedure to process the logic

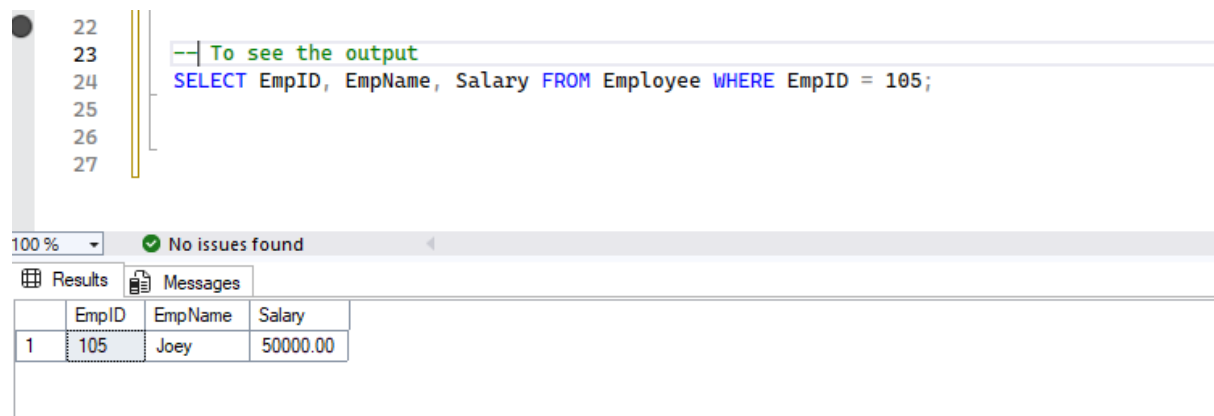
Once we created the Procedure, We have to call that inorder to process the data

```
18
19 -- Call the stored procedure
20 EXEC UpdateSalary @EmpID = 105, @Increment = 5000;
21
22
```

We are updating the salary of employee who has Emp id – 105 .

Step 3- Get the Output

After that to get the result use select statement



```
22
23 -- To see the output
24 SELECT EmpID, EmpName, Salary FROM Employee WHERE EmpID = 105;
25
26
27
```

100 % No issues found

Results

	EmpID	EmpName	Salary
1	105	Joey	50000.00

Function

- Must return a value (scalar or table).
- Usually used for **computations** or reusable logic in queries.
- Cannot perform transactions (in many RDBMS).
- Called inside SQL statements like SELECT function_name(...).

The Function(AnnualSalary) is created for to calculate annual salary of each employee.

The screenshot displays two SQL queries in a SQL Server environment. The first query creates a function named `fn_AnnualSalary` that takes an `EmpID` as input and returns a decimal value representing the annual salary. The second query calls this function for each employee in the `Employee` table, displaying their names and calculated annual salaries.

```
1  -- Function to calculate annual salary
2  CREATE FUNCTION fn_AnnualSalary (@EmpID INT)
3  RETURNS DECIMAL(10,2)
4  AS
5  BEGIN
6      DECLARE @AnnualSalary DECIMAL(10,2);
7
8      SELECT @AnnualSalary = Salary * 12 FROM Employee WHERE EmpID = @EmpID;
9
10     RETURN @AnnualSalary;
11 END;
12
13
```

Messages

Commands completed successfully.

Completion time: 2025-08-19T21:38:53.3070386+05:30

```
12
13 -- Call the function
14 SELECT EmpName, dbo.fn_AnnualSalary(EmpID) AS AnnualSalary FROM Employee;
15
```

Results

	EmpName	AnnualSalary
1	Ross	600000.00
2	Monica	720000.00
3	Chandler	660000.00
4	Rachel	840000.00
5	Joey	600000.00
6	Phoebe	900000.00

Uses

- Filter results
- Compute values
- Create virtual tables

Difference between a procedure and a function?

Procedures:

1. Do not necessarily return a value. They can use IN, OUT, and INOUT parameters to pass data in and out. They are primarily used for performing actions (e.g., updating data, managing transactions). Procedures are called using EXEC or EXECUTE.
2. Can perform actions (INSERT, UPDATE, DELETE), return multiple values, but cannot be used inside SELECT.

Functions:

1. Must return a single value (scalar function) or a table (table-valued function). They can only use IN parameters. They are primarily used for calculations and can be used within SQL statements like SELECT and WHERE.
2. Cannot perform major DML (except table variable updates).

IN/OUT parameter

IN Parameter: The default type. The calling program passes a value to the routine. The routine can use this value but cannot change it in a way that is visible to the caller.

OUT Parameter: The routine passes a value back to the caller. The initial value is null inside the routine. The routine can set a new value for the caller to receive.

INOUT Parameter: The calling program passes a value, and the routine can modify it. The modified value is passed back to the caller.

```
1  --IN & OUT parameters
2  CREATE OR ALTER PROCEDURE Sampleroc
3      @Input INT,          -- IN parameter
4      @Output INT OUTPUT   -- OUT parameter
5  AS
6  BEGIN
7      SET @Output = @Input * 2;
8  END;
9  GO
10
11  DECLARE @Result INT;
12
13  EXEC Sampleroc @Input = 10, @Output = @Result OUTPUT;
14
15  -- Display the result in the Results tab
16  SELECT @Result AS OutputValue;
```

100 % No issues found

Results Messages

	OutputValue
1	20

Can functions return tables?

Yes, in SQL Server, functions can return tables. These are known as Table-Valued Functions (TVFs). They are excellent for creating reusable, parameterized views.

Here we fetched the employee details by department..

```
18
19  -- Create a table-valued function
20  CREATE FUNCTION GetEmployeesByDept (@departmentName VARCHAR(50))
21  RETURNS TABLE
22  AS
23  RETURN (
24      SELECT e.EmpID, e.EmpName, e.Salary, d.DeptName
25      FROM Employee e
26      JOIN Department d ON e.DeptID = d.DeptID
27      WHERE d.DeptName = @departmentName
28  );
29
30  -- Call the table-valued function
31  SELECT * FROM dbo.GetEmployeesByDept('HR');
```

100 % 1 0

Results Messages

	EmpID	EmpName	Salary	DeptName
1	101	Ross	50000.00	HR
2	105	Joey	50000.00	HR
3	106	Phoebe	75000.00	HR

RETURN Statement

RETURN is used in a function to specify the value that the function will return. In a stored procedure, RETURN is used to exit the procedure immediately and can optionally return an integer status code to the caller.

- In functions → return value.
- In procedures → exit procedure (optional return integer).

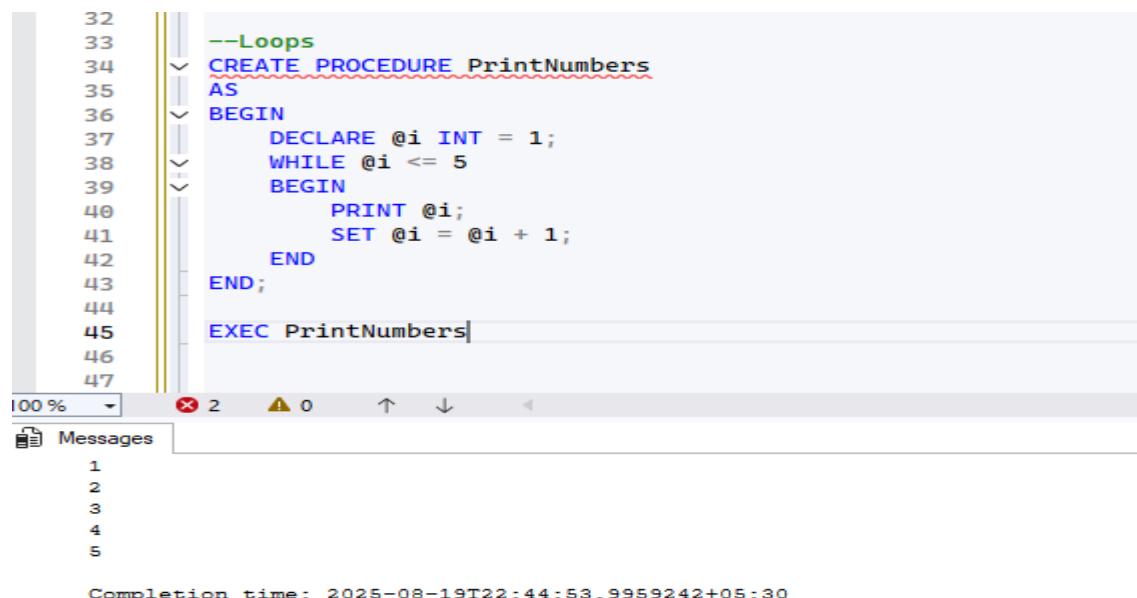
Difference between scalar and table-valued functions?

Scalar Functions: Return a single value (e.g., a number, a string). They can be used anywhere a literal value or expression is valid.

Table-Valued Functions (TVFs): Return a table. They are used in the FROM clause of a query, just like a regular table or view.

Can procedures have loops?

Yes, stored procedures can have loops. SQL Server supports WHILE loops and cursors for iterative processing.



```
32
33
34 --Loops
35 CREATE PROCEDURE PrintNumbers
36 AS
37 BEGIN
38     DECLARE @i INT = 1;
39     WHILE @i <= 5
40     BEGIN
41         PRINT @i;
42         SET @i = @i + 1;
43     END
44 END;
45 EXEC PrintNumbers
46
47
```

100 % 2 0

Messages

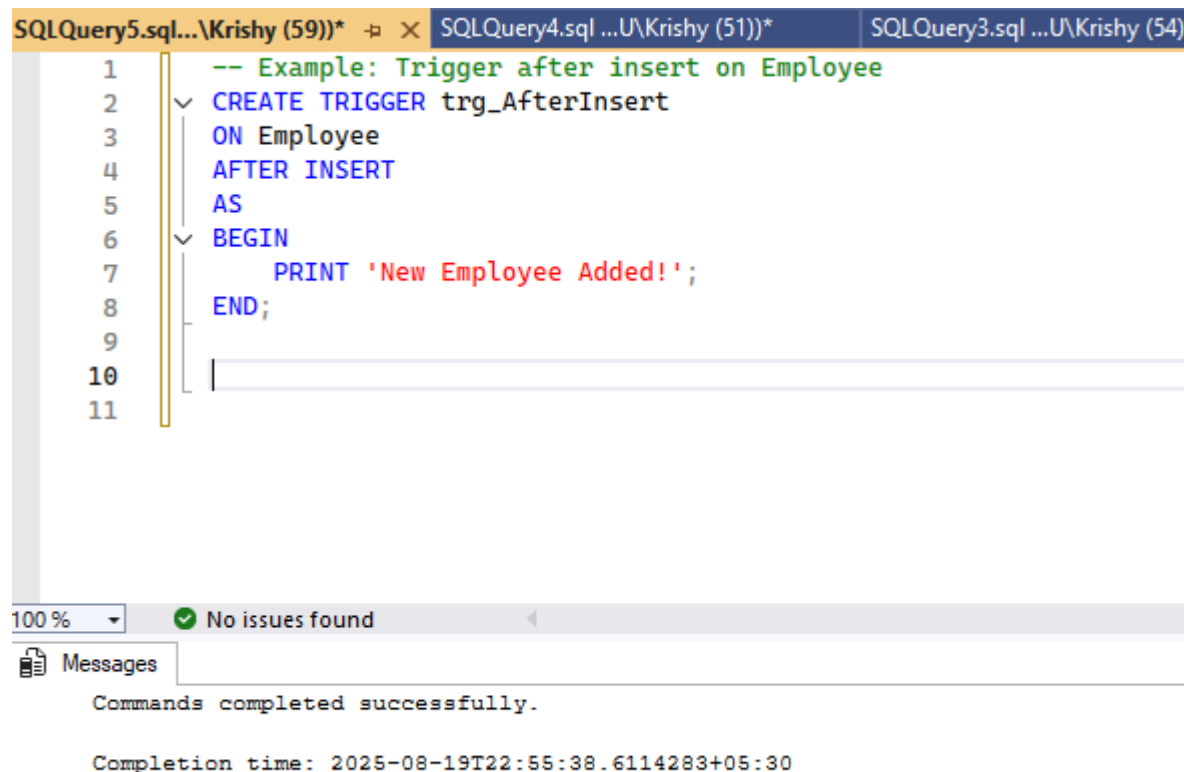
1
2
3
4
5

Completion time: 2025-08-19T22:44:53.9959242+05:30

Triggers

A trigger is a special type of stored procedure that is automatically executed in response to certain events on a table or view, such as INSERT, UPDATE, or DELETE. Triggers are used to enforce business rules, maintain data integrity, or perform logging.

Created a trigger for the insertion of new record, it will automatically trigger the message “New employee Added!” everytime new records added.



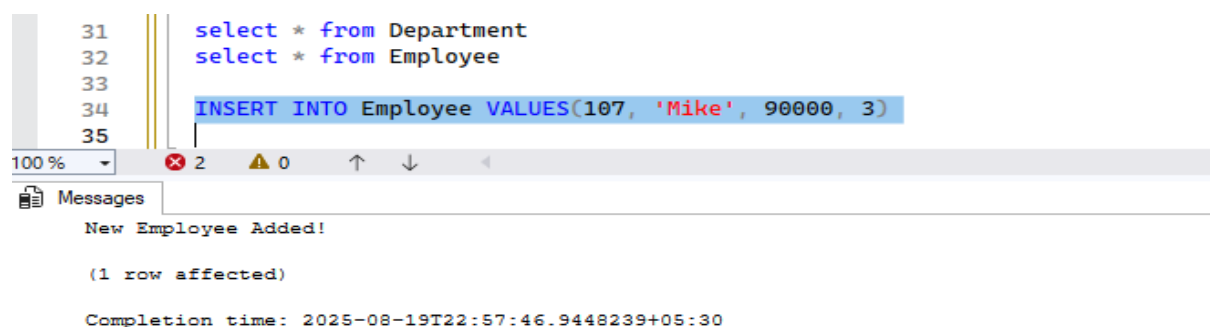
The screenshot shows the SQL Server Enterprise Manager interface. At the top, there are three tabs: 'SQLQuery5.sql... \Krishy (59))', 'SQLQuery4.sql ...U\Krishy (51))', and 'SQLQuery3.sql ...U\Krishy (54)'. The active tab is 'SQLQuery5.sql... \Krishy (59))'. The main area displays a SQL script for creating a trigger. The script is as follows:

```
1  -- Example: Trigger after insert on Employee
2  CREATE TRIGGER trg_AfterInsert
3  ON Employee
4  AFTER INSERT
5  AS
6  BEGIN
7      PRINT 'New Employee Added!';
8  END;
```

Below the script, there is a status bar showing '100 %' and 'No issues found'. At the bottom, there is a 'Messages' pane with the following text:

```
Commands completed successfully.

Completion time: 2025-08-19T22:55:38.6114283+05:30
```



The screenshot shows the SQL Server Enterprise Manager interface. At the top, there are three tabs: 'SQLQuery5.sql... \Krishy (59))', 'SQLQuery4.sql ...U\Krishy (51))', and 'SQLQuery3.sql ...U\Krishy (54)'. The active tab is 'SQLQuery5.sql... \Krishy (59))'. The main area displays a SQL script for inserting a new record into the Employee table. The script is as follows:

```
31  select * from Department
32  select * from Employee
33
34  INSERT INTO Employee VALUES(107, 'Mike', 90000, 3)
35
```

Below the script, there is a status bar showing '100 %' and '2' errors. At the bottom, there is a 'Messages' pane with the following text:

```
New Employee Added!

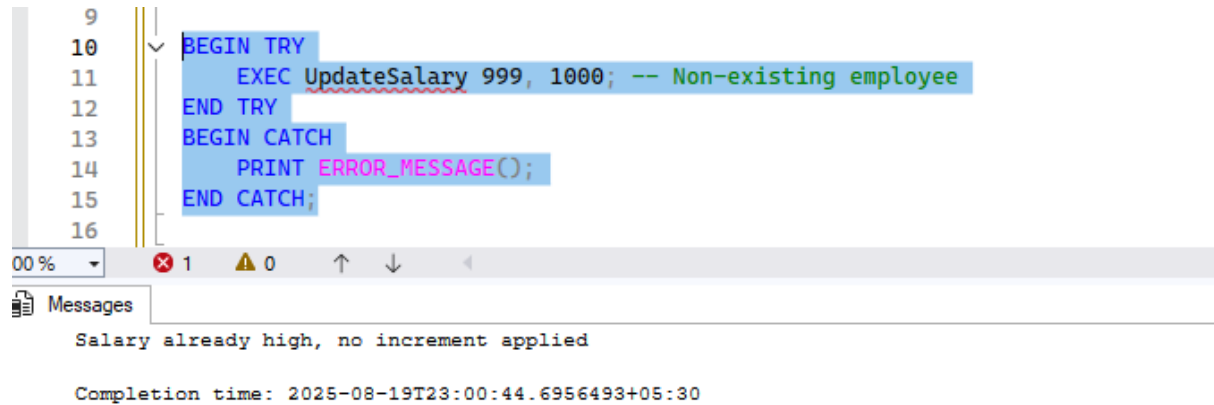
(1 row affected)

Completion time: 2025-08-19T22:57:46.9448239+05:30
```

Here you see the message after new record added.

Debugging stored procedures

- Use PRINT statements.
- Use TRY...CATCH for error handling.



```
9
10 BEGIN TRY
11     EXEC UpdateSalary 999, 1000; -- Non-existing employee
12 END TRY
13 BEGIN CATCH
14     PRINT ERROR_MESSAGE();
15 END CATCH;
16
```

00 % 1 0

Messages

Salary already high, no increment applied

Completion time: 2025-08-19T23:00:44.6956493+05:30

As Salary greater than 60000 it shows 'Salary already high, no increment applied' which we already coded in while creating Stored Procedures UpdateSalary.

```
IF @CurrentSalary < 60000
    UPDATE Employee SET Salary = Salary + @Increment WHERE EmpID
= @EmpID;
ELSE
    PRINT 'Salary already high, no increment applied';
END;
```

Thank you!